

DINOv3-TRT-Acceleration 项目计划报告

项目	信息
报告版本	V1.0
发布日期	2026-04-30
项目代号	DINOv3-TRT-Acceleration
项目性质	PolyU 研究型项目
预计周期	1 个月（2026 年 5 月）
负责人	郑棉鹏

0. 执行摘要

本项目针对 Meta DINOv3 视觉自监督基础模型（ViT-L/16，24 层），构建一条 **PyTorch → ONNX → TensorRT** 的完整推理加速管线，覆盖 **FP32 / FP16 / INT8** 三档精度，并提供 **Python 研究 API** 与 **C++ 生产推理封装** 双栈。

与常规推理加速项目的关键差异：DINOv3 推理输出取 transformer 第 **4 / 12 / 16 / 20** 层中间特征（多尺度密集预测特征），共 4 个 output binding。这一约束贯穿 ONNX 导出、引擎构建、INT8 校准、benchmark、精度对齐与跨语言封装的所有阶段。

关键工程要点：DINOv3 默认含 4 个 **register tokens**（HuggingFace `Dinov3Config.num_register_tokens=4` 硬编码）与 **RoPE 旋转位置编码**——这两点直接影响 ONNX 图结构与 TRT 引擎构建（详见 §2.1、ADR-001、ADR-007）。此外，由于 `IInt8EntropyCalibrator2` 自 TRT 10.1 起 deprecated，主路径采用 NVIDIA TensorRT Model Optimizer 的显式 Q/DQ 量化（详见 ADR-003）。

预期成果：

- FP16 端到端加速 $\geq 1.5\times$ / **stretch 1.8x**（vs PyTorch eager），精度无损
- INT8 端到端加速 $\geq 2.2\times$ / **stretch 2.5x**，下游任务精度跌幅 $\leq 1\%$
- C++ 推理 wrapper 与 Python 数值一致（**分档**：FP32 $\leq 1e-5$ 、FP16 $\leq 1e-3$ 、INT8 cos ≥ 0.99 ）
- 完整 benchmark 矩阵：3 精度 $\times$ 4 batch $\times$ 3 分辨率（含 p50 标准差 $\leq 5\%$ 、中位数去极值统计）
- 实验报告 + 可复现代码

1. 项目背景与目标

1.1 背景

DINOv3 是 Meta 发布的视觉基础模型，在分类、分割、深度估计等下游任务中表现优异，但其推理延迟在生产部署中是瓶颈。NVIDIA TensorRT 提供算子融合、精度量化、kernel 自动调优等加速能力。本项目研究 DINOv3 在 TRT 上的加速极限与精度边界，并形成可复用的工程实践。

1.2 目标 (SMART)

#	目标	度量方式
G 1	将 DINOv3 ViT-L/16 推理延迟降至 PyTorch baseline 的**≤ 67% (FP16, 相当 1.5× 加速); stretch ≤ 56% (1.8×) **	CUDA event p50/p95 测量, 主力机 RTX 5080
G 2	INT8 量化加速**≥ 2.2× (stretch 2.5×) **, 精度退化可量化、可控	4 个输出张量逐层逐输出独立汇报 cosine sim ≥ 0.99; 下游分类 top-1 跌幅 ≤ 1%
G 3	同一 <code>.engine</code> 在 Python 与 C++ runtime 上数值一致	分档: FP32 MaxAbsErr ≤ 1e-5; FP16 ≤ 1e-3; INT8 cos ≥ 0.99
G 4	输出完整 benchmark 矩阵与技术报告	3 精度 × 4 batch × 3 分辨率 CSV + 可视化; 含 p50 std ≤ 5% 与中位数去极值
G 5	代码可复现	单脚本一键复现, README 完整, 附 license 副本与 "Built with DINOv3" 标注

1.3 范围 (In / Out)

In Scope:

- ViT-L/16 (主), ViT-H/14 (如时间允许)
- 4 输出 (layer 4/12/16/20, 即 0-based `[3, 11, 15, 19]`) 的多输出引擎
- FP32 / FP16 / INT8 三档精度
- ImageNet val 子集精度对齐
- Python + C++ 推理 runtime 封装
- 单 GPU 推理 (不含 multi-GPU/分布式)

Out of Scope:

- 模型训练、微调、蒸馏
- 移动端/嵌入式部署 (Jetson/边缘)
- 服务化部署 (K8s / Triton Inference Server)
- 自定义 TRT plugin (仅在原生算子不支持时按需追加)

Stretch Goals (不计入本期范围, 但为后续版本铺路): FP8 PTQ via Model Optimizer (Blackwell 5th-gen Tensor Core)、SmoothQuant ViT-L PTQ、4 层组合 ablation (`[3, 11, 15, 19]` vs `[5, 11, 17, 23]` vs `[4, 12, 16, 20]`)。

2. 关键技术约束

2.1 模型输出规约

DINOv3 ViT-L/16 LVD-1689M 默认含 4 个 **register tokens** (Darcet et al., 2023, "Vision Transformers Need Registers")。完整 token 序列在 224×224 输入下为：

$$[\text{CLS}] + [\text{reg}_1 \text{ reg}_2 \text{ reg}_3 \text{ reg}_4] + [196 \text{ patch tokens}] = 201 \text{ tokens}$$

官方 `get_intermediate_layers()` API 默认裁剪 register tokens，仅返回 `[CLS] + [196 patch] = 197 tokens`。

模式	Token 序列长度	形状	约束
API 默认（裁剪 register）	197	<code>[B, 197, 1024]</code>	本项目主路径
保留 register tokens	201	<code>[B, 201, 1024]</code>	仅当下游任务需要 register 信号时启用

决策：主路径沿用 197 tokens 形状，并在 ADR-001 显式声明"裁剪 register tokens"为 architectural invariant，禁止隐式切换。

DINOv3 推理需输出 transformer 第 4 / 12 / 16 / 20 层（1-based；等价 0-based `[3, 11, 15, 19]`）中间特征：

输出 binding	形状	用途
<code>feat_layer_4</code>	<code>[B, 197, 1024]</code>	浅层语义特征
<code>feat_layer_12</code>	<code>[B, 197, 1024]</code>	中层语义特征
<code>feat_layer_16</code>	<code>[B, 197, 1024]</code>	中高层语义特征
<code>feat_layer_20</code>	<code>[B, 197, 1024]</code>	高层语义特征

来源说明：4/12/16/20 这一组合非 DINOv3 官方 canonical 推荐，源自 DPT 风格密集预测任务的社区惯例（24 层 ViT-L 上常见 [5,11,17,23]，本项目选择偏中后层组合 [4,12,16,20]）。该约束自动排除 ViT-S/B（仅 12 层）。层组合的 ablation 已纳入 stretch goal。

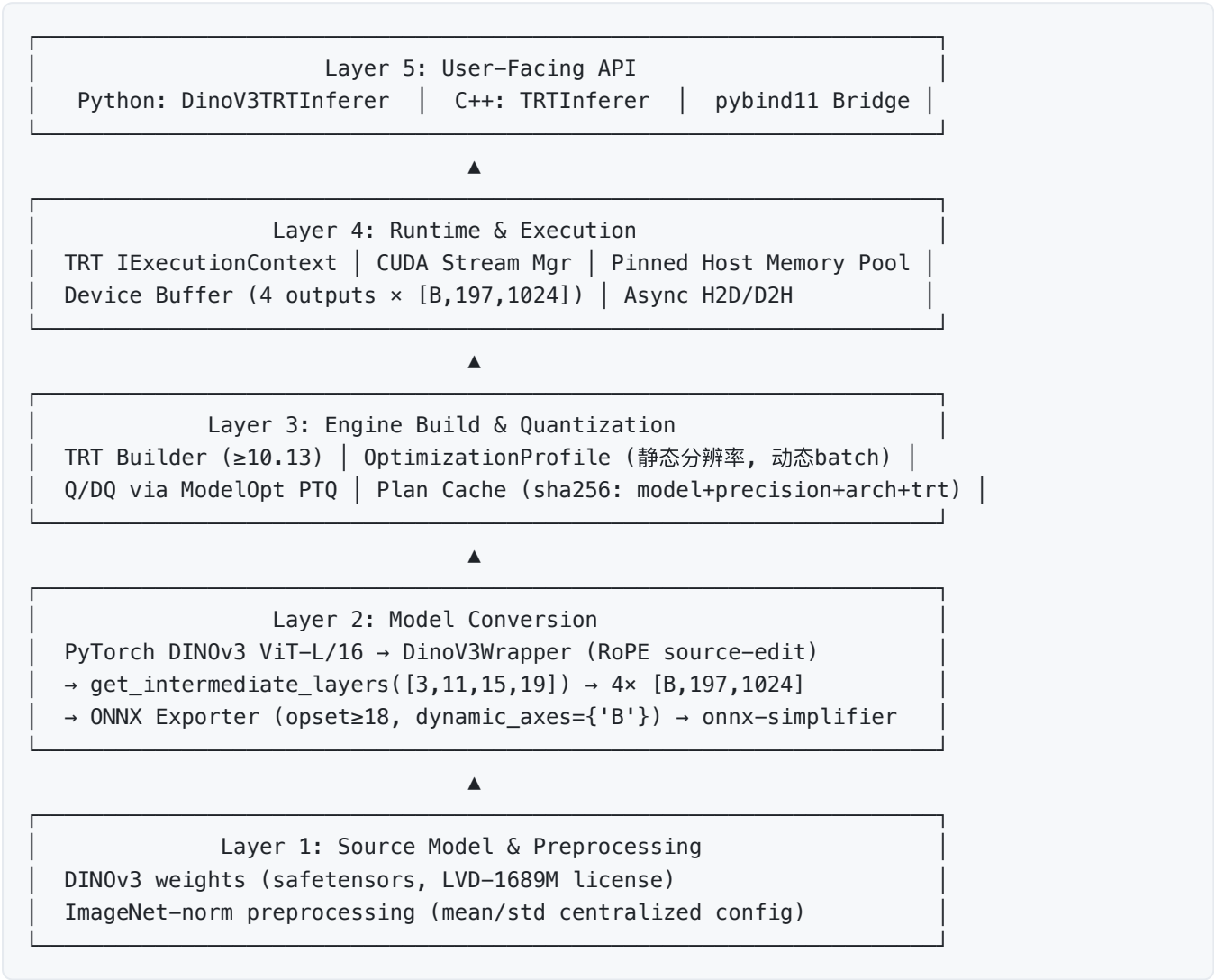
2.2 技术栈版本约束

维度	选型	说明
模型权重	DINOv3 ViT-L/16 LVD-1689M (safetensors 格式)； HuggingFace <code>facebook/dinov3-vitl16-pretrain-lvd1689m</code> （需点击同意 license）	自定义 license / 渠道 / 格式

维度	选型	说明
训练框架	PyTorch 稳定版 cu128 优先 (2.7+ 已 OK)；仅 exporter bug 触发时升 nightly 2.12.x	稳定版已支持 cu128
中间表示	ONNX opset ≥ 18 (RoPE 与 LayerNorm 原生算子需要)	opset 17 RoPE 拆解为细粒度三角函数
推理引擎	TensorRT ≥ 10.13 (理想 10.16.1)；10.8 仅作 baseline 对照	TRT 10.10 已知 16% ViT 回归；10.14.1 修复 55% MHA Blackwell 回归；10.16.1 修复 78% FP8 回归
GPU 计算栈	CUDA ≥ 12.8 + cuDNN 9.x (与主力机 RTX 5080 / Blackwell 匹配)	—
Python runtime	cuda-python (非 pycuda)	—
C++ binding	pybind11	—
量化框架	NVIDIA TensorRT Model Optimizer (Q/DQ 显式量化) 为主路径；Int8EntropyCalibrator2 已 deprecated (TRT 10.1+)，仅作 baseline 对照	官方当前推荐路径
校准数据	ImageNet-1k val 500–1000 张分层抽样，独立于评估集	—
License	DINOv3 License (自定义协议，非 Apache/MIT)，需附 license 副本与 "Built with DINOv3" 标注	—

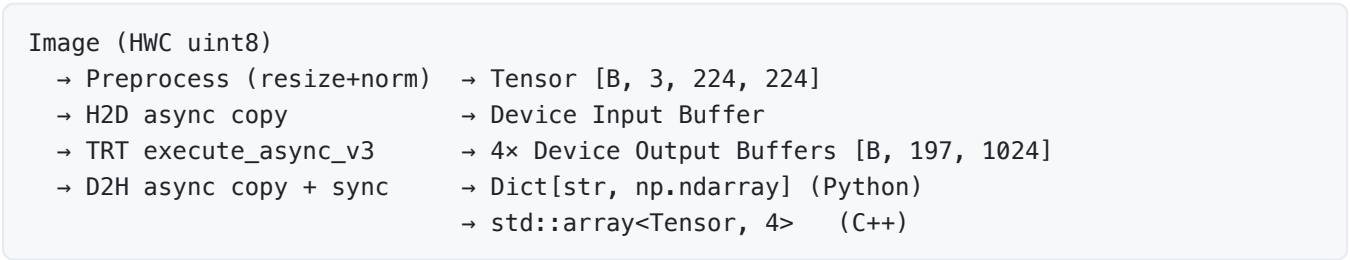
3. 系统架构

3.1 分层架构图



Layer 2 显式标注 RoPE 源码改造 (避免 `If` 节点); Layer 3 标注 Q/DQ 量化路径与版本下限。

3.2 数据流



3.3 模块划分

模块	关键文件/类	职责	依赖
<code>src/export/</code>	<code>onnx_exporter.py</code> , <code>dinov3_wrapper.py</code> , <code>rope_patch.py</code>	DinoV3Wrapper 包装多输出; RoPE 源码改造 (去除 If 节点); <code>torch.onnx.export</code> 配 <code>dynamic_axes</code> (仅 batch); 调用 <code>onnx-simplifier</code> ; schema 验证	<code>torch</code> ≥ 2.7+cu128, <code>onnx</code> ≥ 1.15, <code>onnxsim</code>
<code>src/engine/</code>	<code>engine_builder.py</code> , <code>profile_config.py</code> , <code>plan_cache.py</code> , <code>BuildConfig</code>	ONNX → IBuilderConfig (FP16/INT8 flags) → OptimizationProfile (仅 batch 动态) → 序列化; plan 缓存键 = sha256(onnx + precision + trt_version + gpu_arch)	<code>tensorrt</code> ≥ 10.13
<code>src/infer/</code>	<code>trt_inferer.py</code> : <code>DinoV3TRTInferer</code> , <code>buffer_pool.py</code> , <code>stream_manager.py</code>	引擎反序列化、binding 索引、pinned memory、stream 同步、输出 dict 组装; 线程安全 (每线程独占 IExecutionContext)	<code>cuda-python</code> , <code>numpy</code>
<code>src/quantize/</code>	<code>modelopt_ptq.py</code> (主路径), <code>legacy_calibrator.py</code> (baseline), <code>dataset_loader.py</code>	主路径: ModelOpt 显式 Q/DQ 量化 + ONNX Q/DQ 注入; Baseline 路径: <code>Int8EntropyCalibrator2</code> 对照	<code>nvidia-modelopt</code> , <code>tensorrt</code>
<code>src/benchmark/</code>	<code>bench_runner.py</code> , <code>metrics.py</code> , <code>report_writer.py</code>	暖机 → CUDA event 多次计时 → 中位数去极值统计 → p50/p95/p99/throughput → 逐输出 cosine sim 表 → CSV/JSON	<code>numpy</code> , <code>pandas</code>
<code>src/utils/</code>	<code>preprocess.py</code> , <code>logger.py</code> , <code>config_loader.py</code> , <code>tensor_utils.py</code>	统一图像预处理; ImageNet 归一化常量集中; YAML 配置; 结构化日志	<code>PIL</code> , <code>numpy</code> , <code>pyyaml</code>
<code>cpp/include/</code>	<code>trt_inferer.h</code> , <code>tensor.h</code> , <code>status.h</code>	公共头文件: <code>Tensor</code> POD、 <code>Status</code> 错误码	—
<code>cpp/src/</code>	<code>trt_inferer.cpp</code> , <code>cuda_buffer.cpp</code> , <code>engine_loader.cpp</code>	RAII 封装 ICudaEngine/IExecutionContext, stream 与 device buffer 生命周期	<code>TensorRT</code> ≥ 10.13, <code>CUDA Runtime</code>
<code>cpp/bindings/</code>	<code>pybind_module.cpp</code>	暴露 <code>TRTInferer</code> 至 Python, 零拷贝 <code>numpy</code> ↔ <code>Tensor</code>	<code>pybind11</code>

`src/export/rope_patch.py` 集中处理 DINOv3 RoPE 源码改造 (去除 If 条件分支);
`src/quantize/` 双路径 (ModelOpt 主 + Entropy baseline)。

4. 关键设计决策（ADR）

ADR-001 · ONNX 多输出导出方式

项目	内容
决策点	从 ViT 抓取 4 个中间层输出，含 register tokens 的处理策略
选择	DinoV3Wrapper Module + <code>get_intermediate_layers([3,11,15,19])</code> API；register tokens 默认裁剪（API 默认行为），输出形状统一 <code>[B, 197, 1024]</code>
理由	API 默认裁剪 register 与既定形状契约一致；wrapper 显式 <code>return tuple</code> 让 ONNX 识别 4 个输出；不侵入第三方源（除 RoPE 必要改造，见 ADR-007）
权衡	放弃 register tokens 信号（部分下游任务可能受益）；如未来需要保留 register，需新建 ADR-001b
影响	<code>src/export/dinov3_wrapper.py</code> 设计；输出 binding 命名固定为 <code>feat_layer_{4,12,16,20}</code>

ADR-002 · 动态 shape 策略

项目	内容
决策点	固定 vs 动态
选择	单 profile，仅 batch 维动态（min=1, opt=8, max=32），分辨率固定 224×224
理由	分辨率动态会改变 token 数（197 = 14×14+1+register 裁剪），打破 4 输出形状契约
权衡	放弃多分辨率灵活性
影响	<code>engine_builder</code> 接口；多分辨率需多 profile 重构

ADR-003 · INT8 量化方案

项目	内容
决策点	INT8 量化路径选型
选择	主路径：NVIDIA TensorRT Model Optimizer 显式 Q/DQ 量化（Entropy 算法 + 99.99% Percentile A/B 对照）。Baseline 路径：legacy <code>IInt8EntropyCalibrator2</code> （仅作旧文献复现/对照）
理由	<code>IInt8EntropyCalibrator2</code> 自 TRT 10.1 起已 deprecated；ModelOpt 显式 Q/DQ 是 NVIDIA 官方当前推荐路径，支持 per-channel + per-tensor 混合、敏感层混合精度回退
权衡	ModelOpt 学习曲线略高；放弃 <code>IInt8EntropyCalibrator2</code> 的简洁性；但能获得更好的 ViT 精度（ViT attention logits 长尾对长期 deprecated API 不友好）
影响	<code>src/quantize/modelopt_ptq.py</code> 主实现； <code>legacy_calibrator.py</code> 仅对照；多输出联合校准一次即可（不需为 4 输出分别校准），但精度验收必须逐输出汇报

ADR-004 · Python 推理 Runtime

项目	内容
决策点	pycuda vs cuda-python
选择	cuda-python (NVIDIA 官方)
理由	pycuda 维护放缓；cuda-python 与 TRT 10.x / CUDA 12.x 深度对齐
权衡	API 较底层、boilerplate 多
影响	<code>src/infer/</code> 全部 CUDA 调用

ADR-005 · C++/Python 接口边界

项目	内容
决策点	pybind11 数据交换格式
选择	numpy buffer protocol 零拷贝 + 显式 ownership
理由	避免大 tensor 跨语言深拷贝； <code>py::array_t<float></code> + <code>py::capsule</code> 管理生命周期
权衡	内存生命周期需文档化
影响	<code>cpp/bindings/</code> 须 GPU 调用期间释放 GIL

ADR-006 · 预/后处理位置

项目	内容
决策点	CPU vs CUDA kernel vs TRT 内置
选择	预处理 CPU + 后处理 CPU (本期阶段)
理由	优先保证 Python/C++ 数值位级一致；DINOv3 预处理仅 resize+归一化，非瓶颈
权衡	放弃 CUDA preprocessing 的 ~1-2ms 收益
影响	<code>utils/preprocess.py</code> 与 C++ 等价实现共享同一组归一化常量；后续可迁至 NPP/CV-CUDA

ADR-007 · DINOv3 RoPE 节点处理

项目	内容
决策点	DINOv3 的 RoPE 含 <code>aten::if</code> 条件分支 (DINOv2 不存在)，ONNX 导出后保留为 <code>If</code> 节点，触发 TRT IfConditionalOutputLayer 失败 (Issue #4603, #4558)
选择	首选源码改造 (直接修改 DINOv3 fork 中 RoPE 实现，分支静态化为 eval 路径)；Fallback: <code>onnx-graphsurgeon</code> 在 ONNX 图层做 surgery
理由	源码改造最干净 (一次性，可 review)；graph surgery 易脆，跨 PyTorch/ONNX 版本可能失效

项目	内容
权衡	需维护 DINOv3 fork (含改造 patch)；但 patch 很小 (< 20 行)，可作为 PR 反馈到上游
影响	<code>src/export/rope_patch.py</code> 集中管理 patch； <code>docs/dinov3_fork_patches.md</code> 记录改造点；P2 阶段必须先做 RoPE PoC (M2.1) 再做完整 4 输出导出

ADR-008 · TRT 版本锁定

项目	内容
决策点	TRT 10.x 选哪个小版本
选择	TRT 10.13+ (理想 10.16.1)；固化版本到 <code>requirements.txt</code> 与 Docker 镜像；CI 矩阵覆盖 10.13 / 10.14.1 / 10.16.1 三档
理由	TRT 10.8 首次支持 Blackwell sm_120 但 bug 多；10.10 已知 16% ViT 回归；10.14.1 修复 Blackwell 上 ViT 的 55% MHA regression；10.16.1 修复 78% FP8 + 55% MHA regression
权衡	锁定较新版本，可能与某些遗留环境不兼容；但研究项目允许激进
影响	<code>requirements.txt</code> ， <code>Dockerfile</code> ，CI workflow；版本写入 plan_cache 哈希

ADR-009 · 静态分辨率 + 动态 batch

项目	内容
决策点	是否支持多分辨率引擎
选择	静态分辨率 (224×224) + 动态 batch (B=1/4/8/16/32) 单 profile；多分辨率 (336/518) 作为独立引擎
理由	197 token 形状契约依赖固定分辨率；混合分辨率在单引擎中破坏 token 数恒等式；为 cudaGraph 优化铺平道路
权衡	多分辨率需多引擎，存储/管理成本上升
影响	<code>engine_builder.py</code> profile 配置； <code>benchmark/</code> 矩阵按引擎档位拆分

5. 关键接口契约

5.1 Python 推理 API

```
class DinoV3TRTInferer:
    LAYER_NAMES = ("feat_layer_4", "feat_layer_12",
                   "feat_layer_16", "feat_layer_20")

    def __init__(self, engine_path: str, device_id: int = 0): ...

    def infer(self, img: np.ndarray) -> Dict[str, np.ndarray]:
        """
        Args:
            img: HWC uint8 [H,W,3] OR NCHW float32 [B,3,224,224]
        Returns:
            dict with 4 keys (LAYER_NAMES), each value shape [B,197,1024]
        Invariants:
            - keys 顺序稳定
            - 输出 dtype = np.float32 (无论引擎精度)
            - 单次调用线程安全
            - 形状 [B, 197, 1024] = CLS + 196 patch tokens (register 已裁剪, 见 ADR-001)
        """

    def infer_batch(self, imgs: List[np.ndarray]) -> List[Dict[str, np.ndarray]]: ...
```

5.2 引擎构建器

```
def build_engine(
    onnx_path: str,
    precision: Literal["fp32", "fp16", "int8"],
    profile: ShapeProfile,                    # min/opt/max for input batch dim
    quant_config: Optional[ModelOptQuantConfig] = None, # ModelOpt 主路径
    legacy_calibrator: Optional[IIInt8Calibrator] = None, # baseline 对照
    workspace_gb: int = 4,
    trt_version: str = "10.16.1",            # 显式版本锁定
) -> bytes:
    """Returns serialized engine bytes;
    precision='int8' requires either quant_config (主路径) or legacy_calibrator
    (baseline)."""
```

5.3 C++ 推理 API

```
class TRTInferer {
public:
    explicit TRTInferer(const std::string& engine_path, int device_id = 0);
    Status Infer(const Tensor& input,
                 std::array<Tensor, 4>& outputs);    // 索引 0..3 ↔ layer 4/12/16/20
    const std::array<std::string, 4>& OutputNames() const noexcept;
    // 不变量: outputs[i].shape == {B, 197, 1024}; dtype == float32
};
```

5.4 INT8 量化接口

```
# 主路径: ModelOpt 显式 Q/DQ
import modelopt.torch.quantization as mtq

def quantize_dinov3_modelopt(
    model: nn.Module,
    calib_loader: DataLoader,          # ImageNet val 子集, 500-1000 张
    config: dict = mtq.INT8_DEFAULT_CFG, # Entropy + per-channel weight + per-tensor
                                         activation
    forward_loop: Callable = ...,      # 喂入 calib_loader 跑前向
) -> nn.Module:
    """返回 Q/DQ 注入后的 nn.Module, 可直接 export ONNX → TRT 自动识别 Q/DQ 节点"""

# Baseline 路径 (保留对照, 已 deprecated 但 TRT 10.x 仍支持):
class DinoV3LegacyCalibrator(trt.IInt8EntropyCalibrator2):
    def __init__(self, dataset_dir: str, batch_size: int,
                 cache_file: str, num_samples: int = 1000): ...
    # 注: 必须复用 utils/preprocess.py 同一预处理路径
```

5.5 Benchmark CSV Schema

列	类型	说明
run_id	str	UUID
timestamp	ISO8601	运行时刻
model	str	"dinov3_vit_l16_lvd1689m"
precision	str	fp32 / fp16 / int8
quant_path	str	modelopt / legacy / null
batch_size	int	1 / 4 / 8 / 16 / 32
gpu_arch	str	sm_120 (主力机) / sm_86 / sm_89 / sm_90
trt_version	str	10.13 / 10.14.1 / 10.16.1
latency_p50_ms	float	CUDA event 测量

列	类型	说明
<code>latency_p50_std_ms</code>	float	跨 1000 次的 std, 门禁 $\leq 5\%$
<code>latency_p95_ms</code>	float	
<code>latency_p99_ms</code>	float	
<code>latency_median_trimmed_ms</code>	float	中位数去极值 (剔除最高 5% + 最低 5%)
<code>throughput_imgs_s</code>	float	
<code>gpu_mem_mb</code>	int	nvml 峰值
<code>cos_sim_layer4</code>	float	4 输出逐层独立汇报 (不允许仅给均值)
<code>cos_sim_layer12</code>	float	
<code>cos_sim_layer16</code>	float	
<code>cos_sim_layer20</code>	float	
<code>max_abs_err_layer4</code>	float	4 输出 vs PyTorch FP32 baseline
<code>max_abs_err_layer12</code>	float	
<code>max_abs_err_layer16</code>	float	
<code>max_abs_err_layer20</code>	float	
<code>engine_build_time_s</code>	float	
<code>engine_sha256</code>	str	

6. 跨语言数值一致性策略

措施	实现
共享预处理常量	<code>config/preprocess.yaml</code> 单一来源; Python/C++ 启动加载, 禁止硬编码 mean/std
位级一致性验证	单测: 同一 PNG 经两端预处理后 <code>MaxAbsErr < 1e-7</code> ; CI 强制
统一 stream 语义	两端均用非默认 stream, 调用结束 <code>cudaStreamSynchronize</code> 后再读出
确定性数值	TRT builder <code>KOBEY_PRECISION_CONSTRAINTS</code> + <code>KSTRICT_TYPES</code> + <code>KDISABLE_TF32</code> ; C++ 端 <code>CUBLAS_PEDANTIC_MATH</code>
同一引擎二进制	Python 与 C++ 共享同一份 <code>.engine</code> , 仅 runtime 不同; engine SHA256 写入 benchmark 报告

措施	实现
对齐验证流程	<code>tests/test_cross_lang_parity.py</code> ：随机 10 张图 → Python 推理 → C++ 推理 → 逐输出对比；FP32 阈值 <code>MaxAbsErr ≤ 1e-5</code> (TF32 disabled), FP16 <code>≤ 1e-3</code> , INT8 <code>cos_sim ≥ 0.99</code>
种子与随机性	INT8 校准数据集顺序固定；预处理无随机 augmentation

7. 分阶段路线图

7.1 阶段总览

阶段	名称	时长	入口条件	退出标准
P1	环境准备 & 基线复现	4 天	项目立项	4 层输出可正常抓取 (含 RoPE 单 block PoC)；FP32 baseline 数据落库；Linux WSL2 备选环境就绪
P2	ONNX 导出 & 验证	4 天	P1 完成	4 输出 ONNX 通过 ORT 与 PyTorch 对齐 ($\cos \geq 0.9999$)；RoPE If 节点已消除
P3	TRT FP32/FP16 引擎构建	5 天	P2 ONNX 可用	FP16 端到端加速 $\geq 1.5\times$ (stretch 1.8 $\times$)；逐层 $\cos \geq 0.999$
P4	INT8 PTQ 校准 & 精度调优	6 天	P3 FP16 通过	INT8 引擎逐输出 $\cos \geq 0.99$ ；下游精度跌幅 $\leq 1\%$ (ModelOpt 主路径 + Entropy baseline 对照)
P5	Benchmark 全矩阵 & 报告	4 天	P3+P4 引擎齐全	完整 benchmark CSV (含 std + trimmed median)；瓶颈定位明确
P6	C++ 生产封装	5 天	P5 最优引擎确定	C++ 与 Python 数值一致 (按精度分档)；C++ 吞吐 $\geq$ Python 1.1 $\times$ ；零内存泄漏
P7	文档 & 论文化交付	3 天	P5/P6 完成	终稿报告交付；代码一键复现；附 DINOv3 license 副本

总计：31 天（2026 年 5 月，1 个月）（关键路径：P1 → P2 → P3 → **P4** → P5；P6 可与 P5 末段并行）

7.2 各阶段关键任务

P1 — 环境准备 & 基线复现（4 天）

- CUDA 12.8 / TRT 10.13+ / PyTorch 稳定版 cu128 环境搭建（Windows 主 + Linux WSL2 备选）
- 克隆 dinov3 仓库，加载 ViT-L/16 LVD-1689M 预训练权重（safetensors）
- 运行原始推理验证输出形状（含 register tokens 检查）
- RoPE 单 block PoC (**M2.1 子里程碑**)：源码改造 → 单 block ONNX 导出 → TRT 构建成功
- 实现 multi-layer hook（取 `[3,11,15,19]`，0-based）

6. 准备 ImageNet val 1000 子集（评估）+ 500 子集（INT8 校准，独立抽样）

P2 — ONNX 导出 & 验证（4 天）

1. DinoV3Wrapper 包装 4 输出 + RoPE 改造已应用
2. `torch.onnx.export` opset ≥ 18 , 含 dynamic axes（仅 batch）
3. onnxruntime 数值对齐（逐输出 $\cos \geq 0.9999$ ）
4. onnx-simplifier / Polygraphy 化简（注意保护 multi-output 节点不被剪枝）
5. 多分辨率独立导出版本（224 主，336/518 备选；分辨率间不共享 profile）

P3 — TRT FP32/FP16 引擎构建（5 天）

1. trtexec 构建 FP32 引擎，4 binding 校验
2. FP16 引擎构建 + 逐输出数值对齐
3. Python TRT runtime 封装（cuda-python）
4. layer-wise profiling（trtexec / Nsight Systems / Polygraphy `inspect`）找瓶颈层
5. 多 batch 引擎矩阵（B=1/4/8/16/32）

P4 — INT8 PTQ 校准 & 精度调优（6 天，关键路径瓶颈）

1. 主路径：ModelOpt PTQ + 显式 Q/DQ 注入（INT8_DEFAULT_CFG）
2. Baseline 对照：legacy llnt8EntropyCalibrator2（仅复现）
3. 校准集 500 张分层抽样
4. 逐层 sensitivity analysis（Polygraphy bisect），混合精度回退（敏感层 $\rightarrow$ FP16）
5. 下游任务精度对齐验证（如 linear probing 或 4 输出独立精度表）

P5 — Benchmark 全矩阵 & 报告（4 天）

1. 矩阵：{FP32, FP16, INT8-modelopt, INT8-legacy} $\times$ {B=1,4,8,16,32} $\times$ {224,336,518}
2. 指标：latency p50/p95/p99 + std + trimmed median、throughput、显存、功耗
3. Nsight Systems profiling（含 SM120 kernel 调度图）
4. 与 PyTorch / ORT / ORT-CUDA-EP 对照
5. 数据可视化 + 4 输出独立精度表

P6 — C++ 生产封装（5 天）

1. C++ TRT runtime 类设计（RAII，参考 cyrusbehr/tensorrt-cpp-api）
2. CUDA stream 异步推理 + zero-copy
3. 预/后处理 C++ 等价实现（共享常量）
4. pybind11 binding 对照验证（按精度分档阈值）
5. 多线程并发测试

P7 — 文档交付（3 天）

1. 项目计划报告终稿（归档版）
2. 技术报告（实验 + 4 输出消融分析）

3. README / 复现脚本 + DINOv3 license 副本
4. 代码 review 与清理

8. 里程碑与可交付物

ID	名称	阶段	可交付物	验收标准
M1	Baseline 就绪	P1 末	PyTorch 4 层输出脚本 + 数据集子集	FP32 延迟/精度入库
M2.1 ★	RoPE PoC 通过	P1 末 (与 M1 并行)	单 block ONNX + TRT 构建成功证明	RoPE If 节点消除; TRT 引擎可推理
M2	ONNX 4 输出对齐	P2 末	dinov3_vitl16_4out.onnx (多分辨率)	ORT vs PyTorch逐输出 cos ≥ 0.9999
M3	FP16 加速达标	P3 末	*.fp16.engine + Python runtime	加速比 ≥ 1.5× (stretch 1.8×), 逐输出精度无损
M4	INT8 精度合规	P4 末	*.int8.modelopt.engine + *.int8.legacy.engine + 校准缓存	加速比 ≥ 2.2× (stretch 2.5×), 4 输出逐输出 cos ≥ 0.99
M5	Benchmark 矩阵完成	P5 末	全矩阵 CSV + 性能报告	涵盖 4 引擎 × 5 batch × 3 分辨率; 含 std + trimmed median
M6	C++ 生产可用	P6 末	libdinov3_trt.so / dinov3_trt.dll + demo	数值一致 (分档), 无内存泄漏
M7	项目交付	P7 末	终稿报告 + 可复现仓库 + license 副本	通过 review

9. 工作量与资源

9.1 工作量估算

阶段	Person-Day	假设
P1	5 PD	环境配置 + RoPE PoC 是已知工程黑洞
P2	5 PD	ONNX 4 输出 + simplifier 保护 multi-output 节点需 1-2 轮 debug
P3	6 PD	含 runtime 封装与多 batch 引擎
P4	7 PD	ModelOpt 主路径 + Entropy baseline + 敏感层调优
P5	4 PD	自动化脚本驱动

阶段	Person-Day	假设
P6	6 PD	C++ 封装 + binding 验证（按分档阈值）
P7	3 PD	文档与 review
合计	36 PD	1 人 1 个月（紧） / 2 人半月宽

9.2 资源需求

硬件 · 主力设备：RTX 5080 Windows 工作站

维度	规格
GPU	NVIDIA GeForce RTX 5080 (Blackwell, sm_120, 16 GB VRAM, ~300W TDP)
CPU	Intel Core Ultra 9 285K
主板	Gigabyte Z890 AORUS ELITE WIFI7 ICE
内存	127.5 GB
存储	~1.9 TB NVMe
操作系统	Windows 10 Pro 64 位 (Linux WSL2 作为 sm_120 不稳时的备选)

VRAM 容量约束：16 GB 上限要求 INT8 校准 batch ≤ 8 、benchmark 大 batch (B=32) 单独跑、不能与校准并行；OOM 风险纳入风险登记册 R5/R12（详见 §10.1）。

软件

- 主力机：Python 3.10.10 + **PyTorch 稳定版 cu128 (2.7+)**；nightly 2.12.x 仅 exporter bug 触发时启用
- CUDA ≥ 12.8 , cuDNN 9.x, **TensorRT 10.13+ (理想 10.16.1, 固化版本)**
- NVIDIA TensorRT Model Optimizer** (INT8 主路径)
- onnx ≥ 1.15 , onnxruntime-gpu 1.17+, onnx-simplifier, polygraphy, cuda-python
- C++17, CMake 3.20+, pybind11
- Nsight Systems / Compute (profiling)

数据

- ImageNet val 子集 1000–5000 张 (benchmark)
- INT8 校准子集 500 张 (独立抽样, 不与评估集重叠, 分层采样)
- DINOv3 ViT–L/16 LVD–1689M 官方预训练权重 (safetensors, 含 license 副本)

人力

- 负责人：郑棉鹏 (独立完成 P1–P7)

10. 风险管理

10.1 风险登记册 (Top 13)

#	风险	概率	影响	缓解	应急方案
R1	4 层中间输出导致 ONNX 导出图结构异常 (如 LayerNorm 折叠失败)	中	高	提前 PoC 验证; DinoV3Wrapper module; 保护 multi-output 节点不被 simplifier 剪枝	退化为单输出 + 二次推理获取中间层
R2	INT8 校准在 4 输出张量上精度崩塌	高	高	ModelOpt 主路径 + Entropy baseline 双轨; 分层 sensitivity; 敏感层 FP16 混合精度	INT8/FP16 hybrid; 逐输出 $\cos \geq 0.99$ 阈值放宽至 ≥ 0.97
R3	TRT 版本对 ViT 算子 (GELU、LayerNorm) 支持不全	低	中	锁定 TRT 10.13+	自研 TensorRT plugin (极小概率)
R4	dynamic shape 引擎构建失败或性能不佳	低	中	ADR-009 静态分辨率 + 动态 batch 单 profile	退化为多个固定 shape 引擎
R5	显存不足以构建大 batch / 高分辨率 INT8 引擎 (16 GB VRAM)	中	中	分批 build; workspace ≤ 4 GB; 启用 timing cache	降级 batch 上限, 文档说明
R6	C++ 与 Python 数值不一致 (预处理差异)	中	高	共享预处理常量; 逐 stage 对齐; 按精度分档阈值	暂以 Python 版交付, C++ 列为 future work
R7	DINOv3 权重许可/下载受限影响复现	低	高	确认 license 并镜像权重; 附 license 副本	使用 DINOv2 ViT-L/14 作为兜底 (架构相近, ~2 周重测成本)
R8	单/双人团队进度延期	中	中	周节奏跟踪; P6/P7 可裁剪; stretch 不强制	C++ 封装裁剪为 minimal demo
R9 ★	DINOv3 RoPE If 节点 → TRT 构建失败 (DINOv2 不存在的新坑)	高	高	ADR-007 源码改造 (首选); M2.1 RoPE PoC 前置	onnx-graphsurgeon graph surgery; 极端情况降级 DINOv2
R10 ★	Windows + sm_120 WDDM 100ms 延迟尖峰 → benchmark 不稳	中	中	中位数去极值统计; p50 std $\leq 5\%$ 门禁; 多批次重测	切 Linux WSL2 重跑 benchmark
R11 ★	TRT 版本选错引入 ViT 性能回归 (10.10 已知 16% 回归)	中	中	ADR-008 锁定 10.13+; CI 矩阵覆盖 10.13/10.14.1/10.16.1	回退到上一个稳定版
R12 ★	16 GB VRAM 在 INT8 校准 + 大 batch 同时 OOM	中	中	calibration batch ≤ 8 ; benchmark 大 batch 单独跑	降级到 RTX 4090 / A100 重跑 (如可获得)

#	风险	概率	影响	缓解	应急方案
R13 ★	DINOv3 forward 签名与 torch.jit 不兼容 → torch_tensorrt 不可用 (Issue #206)	高	低	不依赖 torch_tensorrt, 主路径走 ONNX → TRT	onnxruntime CUDA EP 作 backup (性能略差但工程稳)

10.2 关键依赖链

- P2 依赖 P1: multi-layer hook + RoPE 改造决定 ONNX 导出图结构
- M2.1 (RoPE PoC) 是 P1→P2→P3 全链路风险点, 必须前置验证
- P3 依赖 P2: 4 输出 ONNX 是引擎构建输入
- P4 依赖 P3: FP16 通过后才做 INT8 (共享构建脚手架与校准基础设施)
- P5 依赖 P3+P4: 需所有精度引擎齐全
- P6 依赖 P5: 选定最优引擎才封装 C++
- P7 横跨全程, 但终稿依赖 P5/P6
- 关键路径: P1 (含 M2.1) → P2 → P3 → P4 → P5 (INT8 + RoPE 是双瓶颈)

11. 质量保障

11.1 关键质量属性

质量属性	目标值	度量方式	责任模块
Latency (p95)	FP16 ≤ 0.67× FP32 (1.5×; stretch 0.56× = 1.8×) ; INT8 ≤ 0.45× FP32 (2.2×; stretch 0.4× = 2.5×)。B=1, RTX 5080 主力机基线	CUDA event 1000 次推理统计 + std ≤ 5% + trimmed median	engine/ · infer/
Through put	FP16 ≥ 1.7×; INT8 ≥ 2.5× (B=32 vs PyTorch eager)	imgs/sec, benchmark/bench_runner.py	engine/
Numerical Accuracy	逐输出: FP16 cos ≥ 0.999, MaxAbsErr ≤ 1e-2; INT8 cos ≥ 0.99	4 输出独立对比 vs PyTorch FP32 (不允许仅给均值)	quantize/ · benchmark/
Cross-Langugae Parity	分档: FP32 MaxAbsErr ≤ 1e-5 (TF32 disabled); FP16 ≤ 1e-3; INT8 cos ≥ 0.99	test_cross_lang_parity.py CI 门禁	infer/ · cpp/ · bindings/

质量属性	目标值	度量方式	责任模块
Reproducibility	同一 (model, precision, GPU arch, trt_version) 下 .engine 哈希稳定；benchmark 跨次 p50 漂移 $\leq 5\%$	engine SHA256 比对；变异系数监控	<code>engine/pl</code> <code>an_cache.py</code>
Maintainability	模块单文件 < 400 行；单测覆盖率 $\geq 80\%$ ；公共 API 100% 类型注解	pytest-cov, mypy --strict	全模块
Portability	主支持sm_120 (Blackwell, 主力机 RTX 5080)；兼容 sm_86 / sm_89 / sm_90；CUDA ≥ 12.8 / TRT ≥ 10.13 ；Linux 主、Windows 次 (Linux WSL2 作为 sm_120 备选)	CI 矩阵构建；主力机 基线必跑	<code>engine/</code> · <code>cpp/</code>

11.2 质量检查门禁

门禁	触发时机	失败处理
预处理位级一致性测试	每次 commit	block merge
RoPE PoC 通过 (M2.1)	P1 末	block P2 启动
Python $\leftrightarrow$ C++ parity 测试 (分档)	P6 之后每次 commit	block merge
Benchmark p50 std $\leq 5\%$	P5 之后	触发性能调查 (疑 WDDM)
Benchmark 回归 (trimmed median 漂移 $\leq 5\%$)	P5 之后周度	触发性能调查
单测覆盖率 $\geq 80\%$	PR	block merge
Type hint 完整性 (mypy --strict)	PR	warning
<code>polygraphy run --check</code>	P3 之后	block merge

12. 验收标准

12.1 项目完成验收

- ❑ M1–M7 + M2.1 全部达标
- ❑ FP16 端到端加速比 $\geq 1.5\times$ (stretch 1.8 $\times$)，vs PyTorch eager, RTX 5080 主力机, B=1
- ❑ INT8 端到端加速比 $\geq 2.2\times$ (stretch 2.5 $\times$)，4 输出逐输出 cos ≥ 0.99
- ❑ C++ runtime 与 Python runtime 数值一致：FP32 MaxAbsErr $\leq 1e-5$ ；FP16 $\leq 1e-3$ ；INT8 cos ≥ 0.99
- ❑ benchmark 矩阵完整：4 引擎档 (FP32/FP16/INT8–modelopt/INT8–legacy) $\times$ 5 batch $\times$ 3 分辨率 CSV；含 std + trimmed median
- ❑ 技术报告 (实验 + 4 输出独立消融分析 + 可视化)
- ❑ 项目可一键复现 (README + 一键脚本 + DINOv3 license 副本 + "Built with DINOv3" 标注)
- ❑ 单测覆盖率 $\geq 80\%$

☐ 通过最终 review

12.2 阶段性验收（每个 milestone）

每个 M1–M7（含 M2.1）都有独立验收标准（见 §8 表格）。每完成一个 milestone 提交简短验收 memo（≤ 1 页）至 [Wiki/0-项目计划/milestones/](#)。

13. 文档与变更管理

13.1 文档集

文档	路径	状态
项目计划报告（当前）	Wiki/0-项目计划/项目计划报告_对外.md	当前
技术报告（实验）	Wiki/2-技术报告/技术报告_V1.0.0.md	已产出（P5/P6 闭环）
汇报材料（执行摘要）	Wiki/2-技术报告/汇报材料_V1.0.0.md	已产出（答辩用，聚焦决策与关键数字）
复现与许可说明	Wiki/2-技术报告/复现与许可说明_V1.0.0.md	已产出
API 参考	代码内 docstring + 自动生成	持续
复现指南	README.md	P7
项目级指引	CLAUDE.md	已就位
DINOv3 fork 改造记录	docs/dinov3_fork_patches.md	P1 未

13.2 变更记录

版本	日期	变更内容	作者
V1.0	2026-04-30	对外汇报版本发布	郑棉鹏

13.3 后续版本计划

- 项目交付（M7）后归档为终稿版本，含 stretch goal 是否纳入的最终决策

14. 附录

14.1 术语表

术语	含义
DINOv3	Meta DINO 系列第三代视觉自监督基础模型（2025）

术语	含义
ViT-L/16	Vision Transformer Large, patch size 16, 24 层
LVD-1689M	DINOv3 训练数据集（自然图像，16.89 亿样本）
Register Tokens	DINOv3 引入的 4 个可学习辅助 token（Darcet et al. 2023）
RoPE	Rotary Position Embedding；DINOv3 实现含 If 节点
TRT	NVIDIA TensorRT 推理优化引擎
PTQ	Post-Training Quantization（训练后量化）
Q/DQ	Quantize / Dequantize 节点对，显式量化的标准表达
ModelOpt	NVIDIA TensorRT Model Optimizer（INT8 主路径）
Int8EntropyCalibrator2	TRT INT8 KL 散度校准器（自 TRT 10.1 deprecated）
Optimization Profile	TRT 动态 shape 三元组（min/opt/max）
Output Binding	TRT 引擎的输出端口；本项目固定 4 个
CLS token	ViT 序列首位的全局聚合 token
Patch token	由图像 patch 投影得到的特征 token，14×14=196 个
WDDM	Windows Display Driver Model；引入 100ms 量级 GPU 延迟尖峰
sm_120	NVIDIA Blackwell 计算架构代号（RTX 5080/5090）

14.2 参考资料

官方文档

- DINOv3: [facebookresearch/dinov3](https://facebookresearch.github.io/dinov3/)
- TensorRT 文档: docs.nvidia.com/deeplearning/tensorrt/
- TensorRT Model Optimizer: github.com/NVIDIA/TensorRT-Model-Optimizer
- ONNX opset: [onnx/onnx · Operator Schemas](https://onnx.ai/onnx/Operator Schemas)
- INT8 量化白皮书: NVIDIA "8-bit Inference with TensorRT"

Top 5 开源仓库 — 按使用阶段排列

#	仓库	URL	用在阶段	价值
1	DEIMv2	github.com/Intellindust-AI-Lab/DEIMv2	P2-P3	唯一公开 DINOv3 + TRT 完整 workflow
2	TensorRT-Model-Optimizer	github.com/NVIDIA/TensorRT-Model-Optimizer	P4	显式 Q/DQ PTQ（INT8/FP8/NVFP4）权威范例
3	cyrusbehr/tensorrt-cpp-api	github.com/cyrusbehr/tensorrt-cpp-api	P6	TRT 10.x C++ RAII 模板，multi-output + dynamic batch + INT8

#	仓库	URL	用在阶段	价值
4	lightly-ai/lightly-train	github.com/lightly-ai/lightly-train	P5-P6	DINOv3 全任务 ONNX/TRT FP16 商业化交付
5	NVIDIA TensorRT Polygraphy	github.com/NVIDIA/TensorRT/tree/main/tools/Polygraphy	P2-P7 全程	跨语言 parity / INT8 calibration / graph surgery 一站式

End of Report